

R für Spark (SparkR / sparklyr)

Auch R-Nutzer können mit Apache Spark arbeiten. Databricks bietet dafür historisch das Paket **SparkR** an, empfiehlt inzwischen jedoch den Umstieg auf die Alternative **sparklyr**. Diese Seite erklärt die Grundlagen von SparkR: wie man DataFrames erzeugt, damit arbeitet und sogar Machine-Learning-Modelle trainiert.

Wichtiger Hinweis: SparkR gilt ab Databricks Runtime 16.0 als veraltet (deprecated). Databricks empfiehlt, stattdessen das Paket **sparklyr** zu verwenden. Für neue Projekte in R sollte man daher direkt mit sparklyr starten.

Was ist SparkR?

SparkR ist ein R-Paket, das eine leichtgewichtige Schnittstelle bietet, um Apache Spark aus R heraus zu nutzen. Über SparkR lassen sich auch Algorithmen aus MLlib, der Machine-Learning-Bibliothek von Spark, direkt aus R aufrufen – man muss also nicht die Sprache wechseln, um verteiltes maschinelles Lernen zu betreiben.

DataFrames erzeugen

Es gibt drei Hauptwege, in SparkR ein DataFrame (genauer: ein "SparkDataFrame") zu erzeugen:

1. **Aus einem lokalen R-Datenrahmen:** Mit der Funktion `createDataFrame()` lässt sich ein gewöhnlicher R- `data.frame` in ein verteiltes SparkDataFrame umwandeln.
2. **Über die Datenquellen-API:** Mit `read.df()` lassen sich Dateien direkt einlesen, unter anderem in den Formaten CSV, JSON, Text und Parquet.
3. **Über SQL-Abfragen:** Mithilfe von `sql()` in Kombination mit `createOrReplaceTempView()` lässt sich ein DataFrame auch über eine SQL-Abfrage auf eine temporäre View erzeugen.

Arbeiten mit DataFrames in SparkR

- **Spalten und Zeilen auswählen:** Über die Funktionen `select()` und `filter()`.
- **Aggregieren:** Mit `groupBy()` und `count()` lassen sich Daten gruppieren und zählen.

- **Spalten transformieren:** Zum Beispiel durch arithmetische Operationen auf bestehenden Spalten, um neue Spalten zu berechnen.

Machine Learning mit SparkR

SparkR macht die Algorithmen von MLlib auch aus R heraus nutzbar. Ein Beispiel ist die Funktion `glm()` (Generalized Linear Model) für lineare oder logistische Regression. Praktisch: Kategoriale Merkmale (z. B. Text-Kategorien) werden dabei automatisch per One-Hot-Kodierung in numerische Werte umgewandelt, ohne dass man das manuell vorbereiten muss.

Nutzung in Notebooks vs. in Jobs

In interaktiven Notebooks muss man seit Spark 2.2 das Paket explizit laden, bevor man SparkR-Funktionen nutzen kann:

```
library(SparkR)
```

Für Jobs, die über `spark-submit` gestartet werden (also nicht interaktiv im Notebook, sondern als eigenständiges Skript), sind zusätzliche Anpassungen im Code notwendig, damit SparkR korrekt initialisiert wird.

Einordnung für Kursteilnehmer

Wer neu mit R und Spark startet, sollte sich – auch wenn diese Seite noch SparkR im Detail erklärt – direkt mit dem empfohlenen Nachfolger **sparklyr** vertraut machen, da SparkR nicht mehr weiterentwickelt wird. Die grundlegenden Konzepte (DataFrames, Transformationen, MLlib-Integration) sind aber vergleichbar und lassen sich gut übertragen.